

Informe R-Tree - Buscador espacial con consultas por rango y KNN

1 Justificación de la Estructura de Datos

Para consultas espaciales (rango y KNN) sobre puntos geográficos, el R-Tree es la estructura idónea frente a una búsqueda lineal $O(N)$. A diferencia de un B^+ , que organiza claves 1D con orden total sin solapamiento entre nodos, el R-Tree agrupa objetos 2D en *Minimum Bounding Rectangles* (MBRs) jerárquicos que pueden solaparse, permitiendo descartar ramas enteras en dos dimensiones. Además, el B^+ no tiene mecanismo natural para KNN (requeriría conocer el radio de búsqueda a priori), mientras que el R-Tree lo resuelve nativamente con *Best-First Search* y poda por distancia mínima al MBR.

Se implementó un R-Tree clásico con *split cuadrático* ($M = 8$, $m = 4$). Aunque el R*-Tree (con reinserción forzada) produce MBRs más compactos, el split cuadrático ofrece un excelente equilibrio entre calidad de partición y simplicidad. Su lógica es: (1) *PickSeeds*: elegir dos semillas cuyo par maximice el área desperdiciada si se agruparan ($O(M^2)$); (2) *PickNext*: asignar cada entrada restante al grupo cuyo MBR crece menos al incluirla; (3) forzar el mínimo m si un grupo lo requiere. La eliminación también difiere fundamentalmente del B^+ : en vez de fusionar nodos subocupados con hermanos (*merge/redistribution*), el R-Tree extrae todos los objetos del subárbol vaciado y los reinserta desde la raíz (*condenseTree*), evitando la complejidad de buscar y redistribuir entre nodos hermanos.

2 Análisis de Complejidad

Sea N el total de objetos, M la capacidad por nodo y $h \approx \log_M N$ la altura del árbol.

- **Inserción y Split:** Se baja desde la raíz eligiendo en cada nivel el hijo con menor incremento de área (*enlargement*), costando $O(\log_M N)$. El split cuadrático es $O(M^2)$ por la comparación por pares. Construcción total: $O(N \log_M N)$ promedio.
- **Rango (DFS con poda espacial):** Se desciende únicamente a los hijos cuyo MBR intersecta la región consultada; los que no tocan la consulta se descartan sin abrir. A diferencia del B^+ , donde una consulta de rango baja por un solo camino determinístico por clave ordenada sin solapamiento, el R-Tree puede explorar múltiples ramas por el solapamiento 2D, pero evalúa solo $O(M^{h-1} + k)$ entradas, donde k es la cantidad de resultados.
- **KNN (Best-First Search):** Se usa un min-heap inicializado con la raíz, ordenado por la distancia mínima al cuadrado (*mindist*²) del punto de consulta al MBR, que actúa como cota inferior estricta. Se extrae el elemento más cercano: si es un nodo interno, se expanden sus hijos insertándolos en el heap; si es un punto de hoja, se agrega a los resultados. Al obtener k puntos se detiene, pues todo lo restante en el heap está garantizado a mayor distancia. Complejidad: $O(\log N + k \log k)$ promedio, muy inferior al $O(N \log k)$ de la búsqueda lineal.
- **Eliminación:** Localizar la hoja cuesta $O(N)$ en el peor caso (recorrido completo del árbol). La fase de condensación toma $O(\log N)$; reinsertar u huérfanos cuesta $O(u \log N)$.

3 Tiempo de los Experimentos

Se evaluó el rendimiento tomando subconjuntos de $n \in \{1000, 5000, 15000\}$ objetos de `cities15000.txt` (33 906 objetos), promediando 200 consultas aleatorias por configuración.

	Consulta	n	Alt.	Nodos RT	Rev. Lin.	RT (ms)	Lin (ms)
!	Rango 5%	1 000	4	3	1 000	<0.01	<0.01
	Rango 5%	5 000	5	5	5 000	<0.01	<0.01
	Rango 5%	15 000	6	15	15 000	<0.01	<0.01
	Rango 10%	1 000	4	4	1 000	<0.01	<0.01
	Rango 10%	5 000	5	14	5 000	<0.01	<0.01
	Rango 10%	15 000	6	42	15 000	<0.01	<0.01
	KNN $k=1$	15 000	6	10	15 000	<0.01	0.234
	KNN $k=10$	15 000	6	17	15 000	<0.01	0.189
	KNN $k=50$	15 000	6	29	15 000	<0.01	0.157

Table 1: Métricas promedio por consulta. Archivo base: `cities15000.txt` (33 906 objetos).

Los tiempos absolutos del R-Tree son <1 ms en todos los casos porque los datos caben completamente en la caché de la CPU. La verdadera ventaja se observa en la métrica de elementos inspeccionados: para rango 10% con $n = 15 000$, el R-Tree abre solo 42 nodos (≤ 336 entradas evaluadas) frente a 15 000 revisiones lineales, una reducción superior al 97%. Al aumentar el rectángulo del 5% al 10%, los nodos visitados se triplican (15→42), pero la búsqueda lineal siempre revisa N . En KNN, incrementar k de 1 a 50 sube los nodos de 10 a 29, mientras la lineal permanece constante en 15 000. El tiempo de construcción es despreciable (≤ 31 ms). Estos patrones son consistentes en `cities5000.txt` (69 237 objetos) y `cities1000.txt` (170 050 objetos), confirmando que el R-Tree escala sub-linealmente frente al $O(N)$ estricto de la búsqueda lineal.